CODEGUARD

AI-Assisted Source Code Security Analysis and Execution Guard

Hackathon Project Submission Document

Project Repository: https://github.com/shreyash-2006/CodeGuard

Domain: Cybersecurity | Artificial Intelligence | Machine Learning | Developer Tools

---

1. Abstract

Modern software development increasingly relies on third-party code, open-source packages, and AI-powered coding assistants. Although these technologies improve productivity, they also introduce security risks, including credential theft, data exfiltration, destructive commands, obfuscated execution, and malicious instructions targeting AI coding agents.

CodeGuard is a lightweight, hybrid source-code security analysis system designed to identify potentially malicious code before execution. It combines rule-based security detection with machine learning classification to analyze submitted code, estimate its risk, identify threat categories, and provide interpretable security findings.

The proposed solution follows a multi-stage static analysis pipeline involving input preprocessing, language identification, code segmentation, deobfuscation, parsing, security rule evaluation, machine learning inference, and risk-score fusion. The resulting output includes an allow/block verdict, a risk score, detected threat categories, explanatory messages, and flagged source-code lines.

CodeGuard uses a local machine learning model and does not require external large language model API calls during inference. Its API, browser dashboard, batch-analysis capabilities, and guarded execution interface make it suitable for integration into developer workflows, code-submission platforms, and AI-assisted software development environments.

The project's objective is to introduce an additional, explainable security layer between code generation or submission and code execution.

2. Problem Statement

Software applications frequently process code obtained from external sources, user submissions, repositories, and AI-generated outputs. Traditional execution workflows may permit code to run without sufficiently examining its potential behavior.

Existing security approaches face several challenges:

- **Keyword-based detection:** Simple blocklists may miss concealed or modified attack patterns and may flag harmless code containing suspicious keywords.
- **Limited contextual analysis:** The same command or function may be legitimate in one context and dangerous in another.
- **Obfuscated malicious code:** Attackers can disguise commands using encoding, string concatenation, or other transformations.
- **Credential and data exposure:** Code may access sensitive information and transmit it to external destinations.
- **Execution risks:** Destructive commands, resource exhaustion, and unsafe system interactions may cause operational damage.
- **AI-generated code risks:** Automatically generated code may contain insecure operations or instructions intended to manipulate AI coding tools.
- **Deployment complexity:** Heavyweight analysis systems may require external services, additional infrastructure, or expensive inference resources.

There is a need for a lightweight and explainable security analysis layer that combines explicit security rules with learned patterns, produces actionable findings, and can be integrated into existing development systems.

3. Proposed Solution

CodeGuard addresses these challenges through a hybrid static-analysis architecture that combines deterministic security rules and machine learning.

Instead of relying solely on suspicious keywords or an external generative AI service, the system processes code through multiple analysis stages. It attempts to understand the code's structure, uncover certain concealed content, identify suspicious operations, and combine the resulting evidence into a unified risk assessment.

The solution provides the following capabilities:

1. Source-code analysis through an API and web dashboard.
2. Identification of multiple categories of potentially malicious behavior.
3. Language-aware preprocessing, parsing, and deobfuscation.
4. Hybrid detection using rule-based analysis and ML classification.
5. Risk scoring and human-readable explanations.
6. Identification of potentially vulnerable source-code lines.
7. Batch analysis for multiple code snippets.
8. A guarded execution interface that refuses to proceed when the security check blocks the input or fails.
9. Local, CPU-based inference without external LLM API dependency.

The system is intended to function as an additional security control, rather than a replacement for operating-system isolation, secure development practices, or dedicated runtime sandboxing.

4. Objectives

The primary objectives of CodeGuard are:

- To detect potentially malicious behavior in submitted source code before execution.
- To improve detection coverage by combining security rules with machine learning.
- To reduce dependence on simplistic keyword-based filtering.
- To identify suspicious behavior across multiple code and configuration formats.
- To generate interpretable findings that developers can understand and investigate.
- To provide a standardized API for integrating code security checks into external applications.
- To support efficient inference on ordinary CPU-based systems.
- To implement a fail-closed policy in the guarded execution interface.
- To evaluate detection quality using classification metrics and performance measurements.

5. Overall Solution Architecture

CodeGuard follows a sequential analysis pipeline. The pipeline transforms raw source code into structured security evidence and then produces a final decision.

5.1 Input Layer

The system receives source code through its web dashboard, REST API, or programmatic interface.

Input preprocessing normalizes the submitted text and applies size restrictions to reduce the risk of excessively large inputs.

5.2 Language Identification

The system estimates the programming language or content format using heuristic, regex-based scoring.

Language identification helps determine which parsing and detection mechanisms are relevant to the input.

5.3 Code Segmentation

The submitted content is separated into meaningful regions, including code, comments, strings, Markdown code fences, and selected embedded command structures.

This distinction is important because suspicious-looking text inside a comment or string does not necessarily represent an operation that will execute.

5.4 Deobfuscation

The system attempts to uncover certain concealed representations, including Base64-encoded content, hexadecimal escapes, string concatenation, ROT13, URL encoding, reversed strings, and selected shell patterns.

The objective is to expose potentially suspicious content that could otherwise be difficult to recognize during direct text inspection.

5.5 Parsing and Structural Analysis

Language-specific parsing methods are used where supported. Python AST parsing, SQL parsing, YAML parsing, and Markdown parsing help the system examine source structure instead of treating the entire input as unstructured text.

Structural analysis can help distinguish actual operations from similar-looking text and support more context-aware detection.

5.6 Rule-Based Detection

The rule engine applies security checks to identify suspicious operations and combinations of behaviors.

The documented detector modules address areas such as credential access, network activity, destructive operations, injection patterns, resource exhaustion, dependency risks, and suspicious execution behavior.

Some checks use behavioral relationships, such as the potential flow of sensitive or untrusted data toward an external destination.

5.7 Machine Learning Classification

The machine learning component uses scikit-learn-based classification.

The documented model pipeline combines TF-IDF representations, character-level and word-level n-grams, handcrafted numerical features, and calibrated logistic regression classifiers. A specialized model is also documented for AI-targeted instructions.

The classifier provides learned evidence that complements the explicit rules.

5.8 Score Fusion

The system combines rule-based findings and ML predictions to produce category-level risk estimates.

The fusion stage also incorporates configured thresholds and adjustments intended to reduce false positives for benign lookalike code.

5.9 Explanation and Verdict

The final response contains a standardized security assessment, including:

- Allow/block verdict.
- Risk score.
- Detected threat categories.
- Identified code language.
- Human-readable explanation.
- Flagged source-code line numbers.

The structured response allows the dashboard or another application to display findings consistently.

6. Threat Categories Addressed

CodeGuard documents nine major threat categories.

Threat Category	Description
Secret Access	Attempts to access sensitive credentials, tokens, or secrets.
Data Exfiltration	Potential unauthorized transmission of sensitive information.
Destructive Command	Operations that may delete, corrupt, or destroy data.
Obfuscated Execution	Concealed or transformed content associated with suspicious execution.
Persistence or Privilege Escalation	Attempts to maintain access or obtain elevated privileges.
Resource Exhaustion	Behavior that may consume excessive memory, CPU, or network resources.
SSRF and Network Exfiltration	Potential abuse of server-side network access or unauthorized data transmission.
AI-Targeted Instructions	Malicious instructions designed to influence or manipulate AI coding agents.
Dependency Risk	Suspicious software dependencies or potentially dangerous package usage.

These categories provide a structured way to organize findings and help developers understand the type of risk identified.

7. Technical Approach and Methodology

7.1 Hybrid Detection Strategy

The central technical approach is the combination of rule-based analysis and machine learning.

Rule-based detection is useful for explicitly described behaviors and known security patterns. Machine learning can recognize statistical patterns learned from training examples, including patterns that may not correspond to a single fixed keyword.

The two components provide complementary evidence. The system then combines that evidence to generate a unified risk assessment.

7.2 Feature Engineering

The ML pipeline converts source code into numerical representations.

TF-IDF character n-grams capture short sequences of characters, while word n-grams capture patterns involving tokens. Handcrafted numerical features provide additional information about the input.

These features allow conventional machine learning models to operate on source-code text without requiring a large language model.

7.3 Classification

The documented implementation uses calibrated logistic regression classifiers and a one-vs-rest classification strategy for threat categories.

The classifier estimates category-related scores from the extracted features. These predictions are subsequently incorporated into the overall detection pipeline.

7.4 Risk Decision

The fusion and response layers combine detection evidence and apply the configured decision policy.

The documented response contract requires blocked verdicts to contain non-empty categories and a risk score of at least 0.5. Allowed verdicts have no reported categories and a score below that threshold.

The risk score should be interpreted according to the system's scoring policy; it should not automatically be treated as a guaranteed probability of maliciousness.

7.5 Fail-Closed Execution Guard

CodeGuard provides a separate guarded execution interface that checks code before allowing execution to proceed.

When the check blocks the input, times out, or encounters an error, the guarded interface refuses to continue. This avoids treating a failed security check as automatic approval.

The guard is an additional control and does not, by itself, provide complete process isolation or eliminate all runtime risks.

8. Technology Stack

Technology	Purpose
Python	Core implementation and security analysis logic
FastAPI	REST API and request handling
Uvicorn	Application server
scikit-learn	Machine learning classification
TF-IDF	Text feature extraction
Logistic Regression	Classification of suspicious code patterns
NumPy and SciPy	Numerical computation and sparse feature operations
Joblib	Loading and saving trained model artifacts
Pydantic	Data validation and structured API responses
Python AST	Structural analysis of Python code
SQLGlot	SQL parsing
PyYAML	YAML parsing
markdown-it-py	Markdown parsing
HTML, CSS, JavaScript	Web dashboard interface

9. System Interfaces and Integration

CodeGuard exposes its functionality through several interfaces.

9.1 Web Dashboard

The browser-based dashboard allows users to submit code, view security findings, and interact with analysis and demonstration features.

9.2 REST API

The `POST /check` endpoint accepts a code snippet and returns its analysis result in a structured format.

The `POST /batch` endpoint supports analysis of multiple snippets, while `POST /batch/csv` supports batch-oriented evaluation using CSV input.

9.3 Programmatic Guard

The Python guard interface allows applications to check code programmatically and use a guarded execution mechanism.

9.4 Integration Workflow

A typical integration follows this sequence:

1. A user or application generates or receives code.
2. The application submits the code to CodeGuard.
3. CodeGuard performs static analysis and classification.
4. The application receives a structured security result.
5. The application uses the verdict to determine whether the code may proceed.
6. If execution is required, it should occur within an appropriately isolated environment.

This architecture allows CodeGuard to be integrated into code-submission systems, development tools, and AI-assisted coding workflows.

10. Innovation and Differentiating Features

CodeGuard's primary contribution is its integrated security-analysis workflow rather than a single isolated detection algorithm.

Hybrid security analysis: Combines explicit rules with ML-based classification to provide complementary evidence.

Context-aware preprocessing: Uses segmentation, deobfuscation, and structural parsing to improve the quality of subsequent analysis.

Explainable results: Returns categories, reasons, and flagged line numbers instead of providing only a binary classification.

Lightweight deployment: Uses a local scikit-learn model and CPU-based inference rather than requiring an external LLM service.

Integration-ready architecture: Provides REST endpoints, batch analysis, a dashboard, and a programmatic guard.

Fail-closed guarded execution: Refuses to proceed when the guard blocks code or encounters a checking failure.

Broad threat coverage: Organizes detections into nine security categories, covering both conventional code threats and risks involving AI-targeted instructions.

11. Evaluation Strategy and Reported Results

The repository reports evaluation results from a held-out synthetic test set.

Metric	Reported Result
Verdict Precision	1.000
Verdict Recall	0.984
Verdict F1 Score	0.992
Benign False-Positive Rate	0.000
Median Latency (p50)	Approximately 19 ms
95th-Percentile Latency (p95)	Approximately 25 ms

These figures are repository-reported results and should not be interpreted as independently verified real-world performance guarantees.

Precision measures the proportion of predicted malicious cases that are actually malicious. Recall measures the proportion of malicious cases detected by the system. F1 summarizes precision and recall, while the false-positive rate measures how often benign inputs are incorrectly flagged.

For a more comprehensive evaluation, CodeGuard should be tested on independent datasets containing diverse programming styles, unseen attack patterns, benign lookalikes, obfuscated examples, and real-world code samples.

Further testing should also examine category-specific precision and recall, false negatives, calibration quality, resource consumption, and performance across input sizes.

12. Expected Impact and Applications

CodeGuard can provide an additional security layer in environments where code must be screened before execution or integration.

Potential applications include:

- AI-generated code review.
- Online programming and code-submission platforms.

- Developer tools and pre-execution checks.
- Automated security screening in CI/CD pipelines.
- Batch analysis of source-code collections.
- Security checks for scripts and configuration inputs.
- Research into lightweight, interpretable code-security detection.

By returning structured and explainable findings, the system can help developers identify suspicious operations earlier in the development lifecycle.

The expected impact depends on the quality of detection, the deployment environment, and the security controls surrounding the analyzer.

13. Limitations and Risk Considerations

CodeGuard is not a complete security solution.

First, static analysis may fail to identify malicious behavior that depends on runtime state, external resources, dynamic code generation, or unfamiliar attack techniques.

Second, both rule-based detection and machine learning can produce false positives and false negatives.

Third, strong performance on synthetic test data does not guarantee equivalent performance against real-world adversaries.

Fourth, a guarded execution interface is not equivalent to a secure sandbox. Code that is permitted to proceed should still be subject to operating-system isolation, restricted privileges, resource limits, and appropriate monitoring.

Finally, language identification and parser coverage may vary across supported formats and input structures.

These limitations motivate additional independent evaluation and defense-in-depth deployment.

14. Future Scope

The following improvements could extend the project beyond its current documented capabilities.

Expanded evaluation datasets: Incorporate independently sourced and carefully labeled benign and malicious code examples.

Improved behavioral analysis: Extend data-flow tracking and cross-function analysis to identify complex attack paths.

Broader language support: Add deeper parsing and security analysis for additional programming languages and runtime environments.

CI/CD integration: Provide integrations for automated code screening in development pipelines.

Policy customization: Allow organizations to configure threat categories, risk thresholds, and enforcement policies.

Sandbox integration: Connect the guarded workflow to isolated execution environments with restricted network access and filesystem permissions.

Continuous model evaluation: Monitor detection quality and evaluate models against newly emerging attack patterns.

Improved explainability: Provide more precise source-to-sink traces and contextual explanations for each finding.

These are proposed future enhancements and should not be represented as already implemented features.

15. Conclusion

CodeGuard presents a lightweight approach to source-code security analysis by combining static analysis, security rules, machine learning, risk-score fusion, and explainable output.

Its multi-stage pipeline examines code structure, attempts to uncover selected obfuscated content, evaluates potential threats, and returns actionable findings through a standardized interface. The local ML architecture and CPU-based inference make the system suitable for integration into environments that need an additional code-screening layer without depending on external LLM APIs.

The project provides a foundation for further research and development in pre-execution code security, especially for AI-assisted software development and automated code-submission workflows.

Its effectiveness should continue to be evaluated against independent real-world datasets, and its execution guard should be complemented by secure runtime isolation.

Project Repository: https://github.com/shreyash-2006/CodeGuard

Live Dashboard: https://codeguard-bdmu.onrender.com/ui/